



Database Management Systems

Window Functions

Prof. Mahitha G

Department of Computer Science and Engineering

Acknowledgment: Teaching Assistants

What are window functions?

In MySQL, a window function performs a calculation across a set of rows that are related to the current row, without collapsing rows into a single output (unlike GROUP BY).

Window functions let you do calculations across rows, but instead of grouping them like SUM() or AVG() normally does, they let you keep every row and just add extra calculated values.

How are they different from aggregate functions?

While aggregate functions collapse rows into one result, window functions add new values without collapsing rows.

Database Management Systems

Windows Functions in SQL - Example

Sample schema - Departments and Employees table

```
[mysql> select * from departments;
```

dept_id	dept_name
1	HR
2	Sales
3	Engineering

3 rows in set (0.01 sec)

```
[mysql> select * from employees;
```

emp_id	emp_name	department	salary	hire_date
101	Alice	Sales	5000	2020-01-15
102	Bob	Sales	7000	2019-03-12
103	Charlie	Engineering	9000	2018-06-01
104	David	Engineering	8000	2021-07-19
105	Eva	HR	6000	2020-09-30
106	Frank	Sales	7500	2022-02-11

6 rows in set (0.00 sec)

Database Management Systems

Windows Functions in SQL - Example

```
mysql> SELECT department, AVG(salary)
-> FROM employees
-> GROUP BY department;
```

department	AVG(salary)
Sales	6500.0000
Engineering	8500.0000
HR	6000.0000

3 rows in set (0.00 sec)

On executing the query shown using aggregate function AVG we get a single row for each department.

```
mysql> SELECT
-> emp_name, department, salary,
-> AVG(salary) OVER (PARTITION BY department) AS dept_avg_salary
-> FROM employees;
```

emp_name	department	salary	dept_avg_salary
Charlie	Engineering	9000	8500.0000
David	Engineering	8000	8500.0000
Eva	HR	6000	6000.0000
Alice	Sales	5000	6500.0000
Bob	Sales	7000	6500.0000
Frank	Sales	7500	6500.0000

6 rows in set (0.00 sec)

Using window functions, all rows in the employee table are maintained along with average salary of the department they belong to.

Why are window functions required?

Window functions let you analyze data across rows without losing individual rows.

They're required when you want both:

1. Detailed row-level data (every employee, every order, etc.)
2. Aggregates or comparisons across groups (totals, averages, rankings, etc.)

Traditional GROUP BY only gives you summary rows, but window functions give you summary + detail in the same query.

Common use cases of window functions include Ranking & Row Numbers, Running Totals & Cumulative Sums, Moving Averages (Trends), Comparisons to Group Values, Percentiles & Distribution, Lag & Lead (Previous/Next Row Access) etc.

Database Management Systems

Windows Functions in SQL - Types



Types of Window Functions in SQL

- **Aggregate functions with OVER()**
SUM(), AVG(), MIN(), MAX(), COUNT()
→ applied over a partition/window instead of collapsing into one row.
- **Ranking functions**
Some of the ranking functions are ROW_NUMBER() , RANK(), DENSE_RANK()
- **Value functions**
Some of the functions are LEAD(), LAG(), FIRST_VALUE() LAST_VALUE()

Syntax

```
SELECT column_name1, column_name2....  
window_function(column_name2)  
OVER([PARTITION BY column_name1]) AS new_column  
FROM table_name;
```

Key Terms

window_function= any aggregate or ranking function

column_name1 and column_name2 = columns to be selected

column_name2= column on which window function is to be applied

new_column= Name of new column

table_name= Name of table

Database Management Systems

Windows Functions in SQL - Example



Let's now try to understand what this query does in detail:

```
SELECT name, department, salary,AVG(salary)  
OVER (PARTITION BY department)  
AS dept_avg_salary  
FROM employees;
```

1. OVER clause tells SQL to not collapse rows like GROUP BY and instead, calculate an aggregate in a 'window' while keeping all rows visible.
2. PARTITION BY department creates separate "mini-groups" (windows) for each department i.e one window for Sales,one window for Engineering,one window for HR. Within each window, calculations are done.
3. AVG(salary)
Inside each department, SQL calculates the average salary.

1. Aggregate Window Function

Aggregate window functions calculate aggregates over a window of rows while retaining individual rows.

Common aggregate functions include:

SUM(): Sums values within a window.

AVG(): Calculates the average value within a window.

COUNT(): Counts the rows within a window.

MAX(): Returns the maximum value in the window.

MIN(): Returns the minimum value in the window.

Database Management Systems

Aggregate Windows Functions in SQL

Some other examples:

1. To find Maximum salary in each department:

```
SELECT emp_ID, emp_name, Department, Salary, MAX(Salary)
OVER (PARTITION BY Department) AS MaxDeptSalary
FROM Employees;
```

```
mysql> SELECT Emp_ID, emp_name, Department, Salary, MAX(Salary)
[    -> OVER (PARTITION BY Department) AS MaxDeptSalary FROM Employees;
+-----+-----+-----+-----+-----+
| Emp_ID | emp_name | Department | Salary | MaxDeptSalary |
+-----+-----+-----+-----+-----+
| 103    | Charlie  | Engineering | 9000   | 9000          |
| 104    | David    | Engineering | 8000   | 9000          |
| 105    | Eva      | HR          | 6000   | 6000          |
| 101    | Alice    | Sales       | 5000   | 7500          |
| 102    | Bob      | Sales       | 7000   | 7500          |
| 106    | Frank    | Sales       | 7500   | 7500          |
+-----+-----+-----+-----+-----+
6 rows in set (0.04 sec)
```

Database Management Systems

Aggregate Windows Functions in SQL

2. Count of employees in each department:

```
SELECT Emp_ID, emp_name, Department, Salary, COUNT(*)  
OVER (PARTITION BY Department) AS DeptEmployeeCount  
FROM Employees;
```

```
mysql> SELECT Emp_ID, emp_name, Department, Salary,  
-> COUNT(*) OVER (PARTITION BY Department) AS DeptEmployeeCount  
-> FROM Employees;  
+-----+-----+-----+-----+-----+  
| Emp_ID | emp_name | Department | Salary | DeptEmployeeCount |  
+-----+-----+-----+-----+-----+  
| 103 | Charlie | Engineering | 9000 | 2 |  
| 104 | David | Engineering | 8000 | 2 |  
| 105 | Eva | HR | 6000 | 1 |  
| 101 | Alice | Sales | 5000 | 3 |  
| 102 | Bob | Sales | 7000 | 3 |  
| 106 | Frank | Sales | 7500 | 3 |  
+-----+-----+-----+-----+-----+  
6 rows in set (0.03 sec)
```

2. Ranking Window Function

Ranking window functions assign a position (rank/number) to each row **within a partition**, usually based on an ORDER BY clause.

Common Ranking Functions:

- 1) **RANK()** : Assigns ranks to rows with the same value receiving the same rank. If there are ties, the next rank is skipped.
- 2) **DENSE_RANK()** : Similar to RANK(), but ranks are not skipped. If there are ties, the next rank is consecutive.
- 3) **ROW_NUMBER()** : Assigns a unique sequential number to each row (1, 2, 3 ...). No ties, every row has a different number.

Database Management Systems

Ranking Window Functions in SQL

1. RANK() Function

- RANK() is a window function that assigns a rank (1st, 2nd, 3rd, ...) to each row within a partition.
- The ranking is based on the ORDER BY clause inside the window.
- Ties (equal values) get the same rank, and the next rank is skipped. For example: if two rows are tied at rank 1, the next row will be rank 3 (not 2) as shown below.
- **Example:** select emp_name, department, salary, RANK() OVER (PARTITION by department ORDER BY salary DESC) AS rank_in_dept FROM Employees;

```
[mysql> SELECT emp_name, department, salary, RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_in_dept FROM Employees;
```

emp_name	department	salary	rank_in_dept
Charlie	Engineering	9000	1
David	Engineering	8000	2
Eva	HR	6000	1
Bob	Sales	7000	1
Frank	Sales	7000	1
Alice	Sales	5000	3

```
6 rows in set (0.01 sec)
```

Database Management Systems

Ranking Window Functions in SQL

2. DENSE_RANK() Function

- DENSE_RANK() is a window function that also assigns ranks (1st, 2nd, 3rd, ...) to rows within a partition.
- The ranking is based on the ORDER BY clause inside the window.
- Ties (equal values) get the same rank.
- No ranks are skipped, Alice is now numbered 2 instead of 3 when rank was used.
- Example: select emp_name, department, salary, DENSE_RANK() OVER (PARTITION by department ORDER BY salary DESC) AS dense_rank_in_dept FROM Employees;

```
mysql> SELECT emp_name, department, salary,  
-> DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dense_rank_in_dept  
-> FROM Employees;
```

emp_name	department	salary	dense_rank_in_dept
Charlie	Engineering	9000	1
David	Engineering	8000	2
Eva	HR	6000	1
Bob	Sales	7000	1
Frank	Sales	7000	1
Alice	Sales	5000	2

6 rows in set (0.01 sec)

Database Management Systems

Ranking Window Functions in SQL

3. ROW_NUMBER() Function

- The ROW_NUMBER() function assigns a unique sequential number to each row within a partition. The numbering starts at 1 for each partition.
- Unlike RANK() or DENSE_RANK(), there are no ties → every row gets its own distinct number, even if values are equal.
- Example: select emp_name, department, salary, ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) as row_num_in_dept from Employees;

```
mysql> SELECT emp_name, department, salary,  
-> ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num_in_dept  
-> FROM Employees;
```

emp_name	department	salary	row_num_in_dept
Charlie	Engineering	9000	1
David	Engineering	8000	2
Eva	HR	6000	1
Bob	Sales	7000	1
Frank	Sales	7000	2
Alice	Sales	5000	3

```
6 rows in set (0.04 sec)
```

1: LAG() Function: This function lets you look at the value of a column from a previous row within the same partition.

- It is very useful for comparing current row values with earlier ones (like salary growth, stock price change, etc.).
- You can specify how many rows back you want to look (default = 1 row).

General syntax:

`LAG(column_name, offset, default_value)`

`OVER (PARTITION BY partition_column`

`ORDER BY order_column)`

`column_name` is the column you want to peek ahead into.

`offset` is how many rows forward to look (default = 1).

`default_value` is optional, what to return if there's no row that far ahead (default = NULL).

Example: SELECT emp_name, department, salary, LAG(salary, 1) OVER (PARTITION BY department ORDER BY salary DESC) AS prev_salary FROM Employees;

```
mysql> SELECT emp_name, department, salary,  
->      LAG(salary, 1) OVER (PARTITION BY department ORDER BY salary DESC) AS prev_salary  
-> FROM Employees;
```

emp_name	department	salary	prev_salary
Charlie	Engineering	9000	NULL
David	Engineering	8000	9000
Eva	HR	6000	NULL
Bob	Sales	7000	NULL
Frank	Sales	7000	7000
Alice	Sales	5000	7000

6 rows in set (0.05 sec)

Use Cases of LAG():

- Compare current salary with previous salary in the same department.
- Track daily stock price changes by comparing today vs yesterday.
- Calculate differences, growth %, or trends row by row.

2 LEAD() Function

- The LEAD() function lets you look at the value of a column from a future row within the same partition. It is the opposite of LAG().
- Useful for comparing the current row with the *next one* (e.g., predicting trends, seeing who comes after).

General syntax:

```
LEAD(column_name, offset, default_value)  
OVER (PARTITION BY partition_column  
ORDER BY order_column)
```

column_name is the column you want to peek ahead into.

offset is how many rows forward to look (default = 1).

default_value is optional, what to return if there's no row that far ahead (default = NULL).

Example: SELECT emp_name,department,salary, LAG(salary,1) OVER (PARTITION BY department ORDER BY salary DESC) AS prev_salary FROM Employees;

```
mysql> SELECT emp_name, department, salary,  
->      LEAD(salary, 1) OVER (PARTITION BY department ORDER BY salary DESC) AS next_salary  
-> FROM Employees;
```

emp_name	department	salary	next_salary
Charlie	Engineering	9000	8000
David	Engineering	8000	NULL
Eva	HR	6000	NULL
Bob	Sales	7000	7000
Frank	Sales	7000	5000
Alice	Sales	5000	NULL

6 rows in set (0.04 sec)

Use Cases of LEAD():

- See what the next employee's salary is in the same department.
- Calculate salary differences ahead of time (LEAD(salary) - salary).
- Compare today's stock price with tomorrow's price (useful in analytics & predictions).
- Identify transitions (e.g., who replaces whom in a sequence).

Practical Use Cases for Window Functions

Window functions are extremely versatile and can be used in a variety of practical scenarios. Below are some examples of how these functions can be applied in real-world data analysis.

SaleID	EmpID	EmpName	Department	CustomerID	SaleDate	Month	Amount
1	101	Alice	HR	C001	2023-01-05	Jan	1000
2	101	Alice	HR	C002	2023-02-10	Feb	2000
3	101	Alice	HR	C001	2023-03-15	Mar	1500
4	102	Bob	HR	C003	2023-01-20	Jan	1200
5	102	Bob	HR	C001	2023-02-25	Feb	1800
6	103	Charlie	IT	C004	2023-01-10	Jan	7000
7	103	Charlie	IT	C005	2023-02-18	Feb	7500
8	104	David	IT	C006	2023-01-28	Jan	7200

Practical Use Cases for Window Functions

SQL statements to create table and populate the values:

```
CREATE TABLE Sales (SaleID INT PRIMARY KEY, EmpID INT, EmpName VARCHAR(50),  
Department VARCHAR(50), CustomerID VARCHAR(10), SaleDate DATE, Month VARCHAR(10),  
Amount DECIMAL(10,2));
```

```
INSERT INTO Sales (SaleID, EmpID, EmpName, Department, CustomerID, SaleDate, Month,  
Amount)
```

```
VALUES
```

```
(1, 101, 'Alice', 'HR', 'C001', '2023-01-05', 'Jan', 1000),  
(2, 101, 'Alice', 'HR', 'C002', '2023-02-10', 'Feb', 2000),  
(3, 101, 'Alice', 'HR', 'C001', '2023-03-15', 'Mar', 1500),  
(4, 102, 'Bob', 'HR', 'C003', '2023-01-20', 'Jan', 1200),  
(5, 102, 'Bob', 'HR', 'C001', '2023-02-25', 'Feb', 1800),  
(6, 103, 'Charlie', 'IT', 'C004', '2023-01-10', 'Jan', 7000),  
(7, 103, 'Charlie', 'IT', 'C005', '2023-02-18', 'Feb', 7500),  
(8, 104, 'David', 'IT', 'C006', '2023-01-28', 'Jan', 7200);
```

1. Retrieve top 2 Sales by each Employee and rank them.

```
SELECT * FROM ( SELECT *, RANK() OVER (PARTITION BY EmpID ORDER BY Amount  
DESC) AS SaleRank FROM Sales ) AS ranked WHERE SaleRank <= 2;
```

```
[mysql> SELECT * FROM ( SELECT *, RANK() OVER (PARTITION BY EmpID ORDER BY Amount DESC) AS SaleRank FROM Sales ) AS ranked WHERE SaleRank <= 2;
```

SaleID	EmpID	EmpName	Department	CustomerID	SaleDate	Month	Amount	SaleRank
2	101	Alice	HR	C002	2023-02-10	Feb	2000	1
3	101	Alice	HR	C001	2023-03-15	Mar	1500	2
5	102	Bob	HR	C001	2023-02-25	Feb	1800	1
4	102	Bob	HR	C003	2023-01-20	Jan	1200	2
7	103	Charlie	IT	C005	2023-02-18	Feb	7500	1
6	103	Charlie	IT	C004	2023-01-10	Jan	7000	2
8	104	David	IT	C006	2023-01-28	Jan	7200	1

```
7 rows in set (0.01 sec)
```

2. Running Total of Sales per Employee

SELECT EmpID, EmpName, Month, Amount, SUM(Amount) OVER (PARTITION BY EmpID ORDER BY SaleDate) AS RunningTotal FROM Sales;

```
[mysql> SELECT EmpID, EmpName, Month, Amount, SUM(Amount) OVER (PARTITION BY EmpID ORDER BY SaleDate) AS RunningTotal FROM Sales;
```

EmpID	EmpName	Month	Amount	RunningTotal
101	Alice	Jan	1000	1000
101	Alice	Feb	2000	3000
101	Alice	Mar	1500	4500
102	Bob	Jan	1200	1200
102	Bob	Feb	1800	3000
103	Charlie	Jan	7000	7000
103	Charlie	Feb	7500	14500
104	David	Jan	7200	7200

```
8 rows in set (0.00 sec)
```

3. First Sale of Each Customer

SELECT * FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY SaleDate) AS rn FROM Sales) AS first_sales WHERE rn = 1;

```
[mysql> SELECT * FROM ( SELECT *, ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY SaleDate) AS rn FROM Sales ) AS first_sales WHERE rn = 1;
```

SaleID	EmpID	EmpName	Department	CustomerID	SaleDate	Month	Amount	rn
1	101	Alice	HR	C001	2023-01-05	Jan	1000	1
2	101	Alice	HR	C002	2023-02-10	Feb	2000	1
4	102	Bob	HR	C003	2023-01-20	Jan	1200	1
6	103	Charlie	IT	C004	2023-01-10	Jan	7000	1
7	103	Charlie	IT	C005	2023-02-18	Feb	7500	1
8	104	David	IT	C006	2023-01-28	Jan	7200	1

```
6 rows in set (0.01 sec)
```

4. Compare Current Sale with Previous Sale (LAG)

SELECT EmpID, EmpName, Month, Amount, LAG(Amount) OVER (PARTITION BY EmpID ORDER BY SaleDate) AS PrevAmount FROM Sales;

```
[mysql> SELECT EmpID, EmpName, Month, Amount, LAG(Amount) OVER (PARTITION BY EmpID ORDER BY SaleDate) AS PrevAmount FROM Sales;
```

EmpID	EmpName	Month	Amount	PrevAmount
101	Alice	Jan	1000	NULL
101	Alice	Feb	2000	1000
101	Alice	Mar	1500	2000
102	Bob	Jan	1200	NULL
102	Bob	Feb	1800	1200
103	Charlie	Jan	7000	NULL
103	Charlie	Feb	7500	7000
104	David	Jan	7200	NULL

```
8 rows in set (0.00 sec)
```

5. Rank Employees by Sale Amount within Department

SELECT EmpID, EmpName, Department, Amount, RANK() OVER (PARTITION BY Department ORDER BY Amount DESC) AS RankInDept FROM Sales;

```
[mysql> SELECT EmpID, EmpName, Department, Amount, RANK() OVER (PARTITION BY Department ORDER BY Amount DESC) AS RankInDept FROM Sales;
```

EmpID	EmpName	Department	Amount	RankInDept
101	Alice	HR	2000	1
102	Bob	HR	1800	2
101	Alice	HR	1500	3
102	Bob	HR	1200	4
101	Alice	HR	1000	5
103	Charlie	IT	7500	1
104	David	IT	7200	2
103	Charlie	IT	7000	3

```
8 rows in set (0.00 sec)
```

- 1) MySQL Window Functions Reference
- 2) SQL Server Window Functions (Microsoft Docs)
- 3) Geeks for Geeks



THANK YOU

Mahitha G

Department of Computer Science and Engineering

mahithag@pes.edu